

Name: Syed Saud Ali

Project: Fraud Detection Analysis for Google Capstone Project in Coursera

Dataset: <https://www.kaggle.com/datasets/ealaxi/paysim1>

Date: 17th August 2026

The Ask Phase

This analysis is divided into 6 phases. The first phase is the ask phase, where I will identify what problem I want to solve, identify my stakeholders and my audience and how my insights will influence my clients to make better decisions.

After analyzing the dataset, I have identified the business task, stakeholders and the metric as seen below:

Business Task:

A fictional mobile payment company has noticed the rise of fraud in their services. They want to know which transaction types and behaviours carry the most risk of fraud so they can crack down on them and prevent them from happening again.

Stakeholders:

Head of Fraud Detection Department of the fictional mobile payment company, who will use these findings to see which fraud patterns to target. Moreover, the personnel working in customer support who handle fraudulent transactions who would benefit from knowing which transaction to review. Audience: Leadership that are not very technical, so it should be easy for them to understand.

Metric:

1. Fraud Rate in % for each transaction type.
2. Fraud detection rate in % for all fraudulent transactions that were flagged by the existing flagging system.

The Prepare Phase

The next phase is the prepare phase, where the task is to find data that is appropriate for this analysis, which I have already found with the PaySim dataset that is linked above. The dataset can be found in kaggle, where it is split into 11 columns:

1. step - 1 unit of time in the real world where 1 step equals to 1 real hour.
2. type - the transaction type.
3. amount - the amount in the transaction.
4. nameOrig - the name of the customer who started the transaction.
5. oldbalanceOrig - initial balance before the transaction.
6. newbalanceOrig - new balance after the transaction.
7. nameDest - the name of customer who is the recipient of the transaction.
8. oldbalanceDest - initial balance of the recipient before the transaction.
9. newbalanceDest - new balance recipient after the transaction.
10. isFraud - flags transactions made by fraudulent agents in the simulation.

11. isFlaggedFraud - where the system flags a transaction as fraudulent.

In terms of data credibility, the dataset is synthetic so it may not perfectly reflect real-world behavior. Below is how the dataset is assessed against ROCCC:

Reliable: data generated is synthetic instead of pulling data from actual transactions so it may not reflect real world behaviors.

Original: it is a secondary source as it is obtained from kaggle as it is not collected first-hand.

Comprehensive: it has over 6 million rows with 11 columns, so the data is very detailed though it lacks date and time data types.

Current: since it was last updated 9 years ago, the data does not represent current trends.

Cited: it is licensed by CC BY-SA 4.0, which means I am free to use and adapt the source as long as the original source is provided. I have stated the source at the beginning of this jupyter notebook.

As seen in the code below, the data's integrity is verified by checking if there were any duplicate or null values, where none were found. I also checked if the values in the isFlaggedFraud and isFraud columns only contain 0 and 1 values to ensure that it only contain valid values as 2 values are needed to show if whether the transaction was flagged as true or false. It helps to answer my question as it can help to identify how fraud detection can be improved. The main problem with the data is that it is very big, having more than 6 million rows. Because of that, the data cannot be rendered in Excel, so the data had to be imported in a postgresql database, which will be touched upon more in the next phase. Other than the fact that the data is simulated instead of using actual data, 2 limitations to the dataset are fraudulent transactions make up around 0.13% of the total transactions and that it only occurs on cash-out and transfer transaction types. Finally, there is no date data type listed in this dataset, so I won't be able to analyze the fraud trends over time.

```
In [8]: import pandas as pd;
import matplotlib.pyplot as plt;
import os;
from dotenv import load_dotenv;
import seaborn as sns;
### This is to connect to PostgreSQL database
from sqlalchemy import create_engine;
### reads the env file
load_dotenv();
### Connect to the database
engine=create_engine(f'postgresql://postgres:{os.getenv("DB_PASSWORD")}@localhost:5432/financial_analysis')
### Select table from the database
df_table=pd.read_sql("SELECT * FROM financial_analysis ORDER BY step ASC",engine)
print(df_table);
### Check if there is any duplicates or null values
print(f'The total number of null values in the filtered data is {df_table.isnull().sum().sum()}')
print(f'The total number of duplicated values in the filtered data is {df_table.duplicated().sum()}')
### Confirm that the isFlaggedFraud and isFraud columns only showing values that are 0 or 1
print(f'Unique values in the isFlaggedFraud column is {df_table["isflaggedfraud"].unique().tolist()}')
print(f'Unique values in the isFraud column is {df_table["isfraud"].unique().tolist()}')
### Identify problems with the data
print(f'Fraudulent transactions make up {(df_table["isfraud"].mean()*100).round(2)}% of the total transactions')
```

```
transactionFraudType=pd.read_sql("SELECT DISTINCT type FROM financial_analysis W
print(f'List of transaction type where fraud has occurred: {list(transactionFrau
```

	step	type	amount	nameorig	oldbalanceorg	\
0	1	CASH_OUT	181.00	C840083671	181.00	
1	1	PAYMENT	11668.14	C2048537720	41554.00	
2	1	PAYMENT	7817.71	C90045638	53860.00	
3	1	PAYMENT	7107.77	C154988899	183195.00	
4	1	PAYMENT	7861.64	C1912850431	176087.23	
...	...	...	...	...	...	
6362615	743	CASH_OUT	339682.13	C786484425	339682.13	
6362616	743	TRANSFER	6311409.28	C1529008245	6311409.28	
6362617	743	CASH_OUT	6311409.28	C1162922333	6311409.28	
6362618	743	TRANSFER	850002.52	C1685995037	850002.52	
6362619	743	CASH_OUT	850002.52	C1280323807	850002.52	

	newbalanceorg	namedest	oldbalancedest	newbalancedest	isfraud	\
0	0.00	C38997010	21182.00	0.00	1	
1	29885.86	M1230701703	0.00	0.00	0	
2	46042.29	M573487274	0.00	0.00	0	
3	176087.23	M408069119	0.00	0.00	0	
4	168225.59	M633326333	0.00	0.00	0	
...	...	...	...	...	...	
6362615	0.00	C776919290	0.00	339682.13	1	
6362616	0.00	C1881841831	0.00	0.00	1	
6362617	0.00	C1365125890	68488.84	6379898.11	1	
6362618	0.00	C2080388513	0.00	0.00	1	
6362619	0.00	C873221189	6510099.11	7360101.63	1	

	isflaggedfraud
0	0
1	0
2	0
3	0
4	0
...	...
6362615	0
6362616	0
6362617	0
6362618	0
6362619	0

[6362620 rows x 11 columns]

The total number of null values in the filtered data is 0.

The total number of duplicated values in the filtered data is 0.

Unique values in the isFlaggedFraud column is [0 1].

Unique values in the isFraud column is [1 0].

Fraudulent transactions make up 0.13% of the total transactions in the dataset.

List of transaction type where fraud has occurred: ['CASH_OUT', 'TRANSFER']

The Process Phase

During this analysis, I used PostgreSQL, Python and Jupyter notebook. PostgreSQL was necessary because it allowed me to store the dataset as a table, since it has over 6 million rows, which is beyond what Excel can handle. Python lets me query the database directly and run calculations using libraries such as pandas and sqlalchemy, which are beneficial for my data analysis. Jupyter helps me store my Python code as well as my markdown explanations so the result and the process can be shown in a single file to make

following the analysis easier. Before starting the analysis I have ensured data integrity by checking for any null or duplicate values in the dataset, with none found in either case, as shown in the output above. I also checked that the isFraud and isFlaggedFraud columns only have 2 values (0 or 1) to ensure that there is no invalid data before I continue on with my analysis. When I imported the data into the PostgreSQL database, the columns were all standardized into lowercase letters to ensure consistency during sql queries.

The Analyze Phase

The data should be organised by the step value in ascending order, which was already done when the data was being gathered by selecting the table as seen in the select statement above. After checking, the data seems to be formatted correctly, as since the data is imported through the database, I created it to only accept a specific data type for each column. Below outlines the code that helps me explore the data:

```
In [9]: ### Find the number of rows and columns in the dataset
print(f'This dataset has {df_table.shape[0]} rows of data and {df_table.shape[1]}
### Find the column names and their data type in the dataset
for column, dtype in zip(df_table.columns, df_table.dtypes):
    print(f'{column} Column: {dtype} Data Type');
### Find a more descriptive statistical summary for the amount column in the dat
print(f'This is the descriptive statistical summary for the amount column in the
```

This dataset has 6362620 rows of data and 11 columns of data.

step Column: int64 Data Type

type Column: object Data Type

amount Column: float64 Data Type

nameorig Column: object Data Type

oldbalanceorg Column: float64 Data Type

newbalanceorg Column: float64 Data Type

namedest Column: object Data Type

oldbalancedest Column: float64 Data Type

newbalancedest Column: float64 Data Type

isfraud Column: int64 Data Type

isflaggedfraud Column: int64 Data Type

This is the descriptive statistical summary for the amount column in the dataset:

count 6.362620e+06

mean 1.798619e+05

std 6.038582e+05

min 0.000000e+00

25% 1.338957e+04

50% 7.487194e+04

75% 2.087215e+05

max 9.244552e+07

Name: amount, dtype: float64

```
In [10]: ### Finding 1: How often does fraud occurs in each transaction type?
fraudTransactionFinding=pd.read_sql('SELECT ROUND(AVG(isFraud)*100,2), type FROM
### Reason for this is because % sign does not work in the read sql function
fraudTransactionFinding.columns = ["Fraud rate (%)", "Transaction type"];
print(fraudTransactionFinding);
### Finding 2: How well does the fraud flagging system actually flag fraudulent
df_fraud=df_table[df_table['isfraud']==1];
fraudDetectionRate=df_fraud['isflaggedfraud'].mean()*100;
print(f'Fraud Detection Rate is {fraudDetectionRate.round(2)}%.');
### Finding 3: How does fraud correlate to an account being drained?
```

```
df_table['emptybalance']=df_table['newbalanceorg']==0; # Find out if the new bal
emptyBalanceRate=df_table.groupby('isfraud')['emptybalance'].mean()*100;
print(f'The rate of legitimate transactions that result in an empty balance is {
print(f'The rate of fraudulent transactions that result in an empty balance is {
# Formula for the emptyBalanceRate works like this:
# (total fraud transactions where new balance after transaction is 0 / total fra
```

	Fraud rate (%)	Transaction type
0	0.00	CASH_IN
1	0.18	CASH_OUT
2	0.00	DEBIT
3	0.00	PAYMENT
4	0.77	TRANSFER

Fraud Detection Rate is 0.19%.

The rate of legitimate transactions that result in an empty balance is 56.68%.

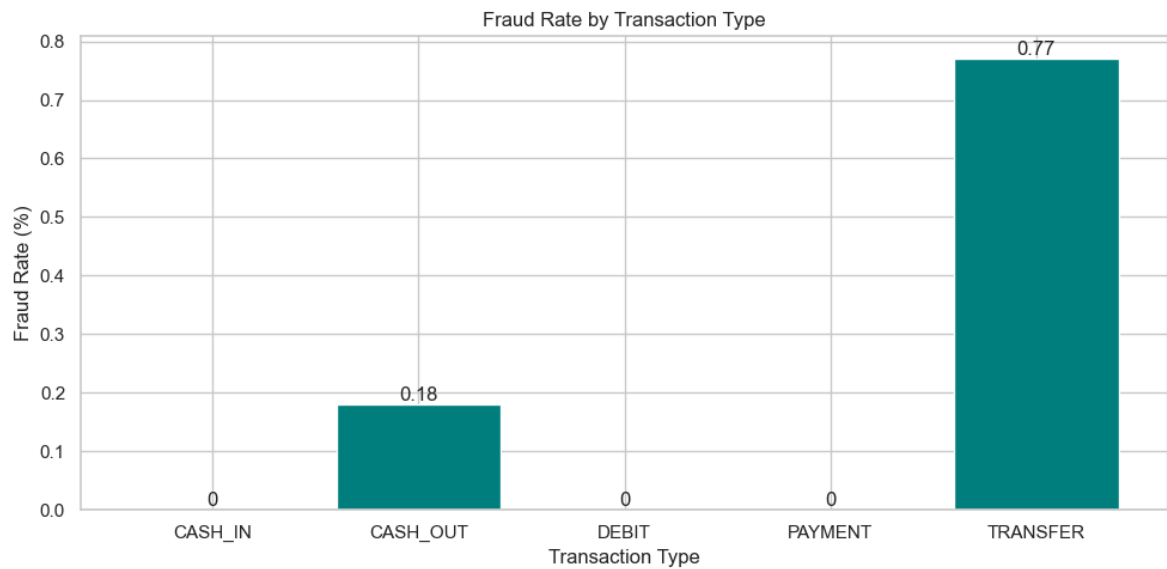
The rate of fraudulent transactions that result in an empty balance is 98.05%.

Now this is where I gather some findings. The first finding was to see the percentage of the transactions split by its type that were identified as fraud. As stated before, only cash_out and transfer types were found to include fraudulent transactions, with cash_out having around 0.18% while transfers have up to 4 times as many fraudulent transactions with almost 0.8%. For the second finding, the fraud flagging system is shown to accurately flag around 0.19% of the total fraudulent transactions as fraud, which is a critical vulnerability for the head of the fraud handling department, since this suggests that the logic behind the fraud flagging system needs to be improved, otherwise these fraudulent transactions won't be investigated by the team. Finally, the last finding is to identify if there was any relationship between fraudulent transactions and whether the account balance after the transaction has been made is 0. As seen in the output above, the result of legitimate transactions that result in an empty balance is more than 56%, while the result of fraudulent transactions that result in an empty balance is over 98%, which shows that fraudulent transactions are over 1.5 times more likely to empty their balance compared to legitimate transactions.

The Share Phase

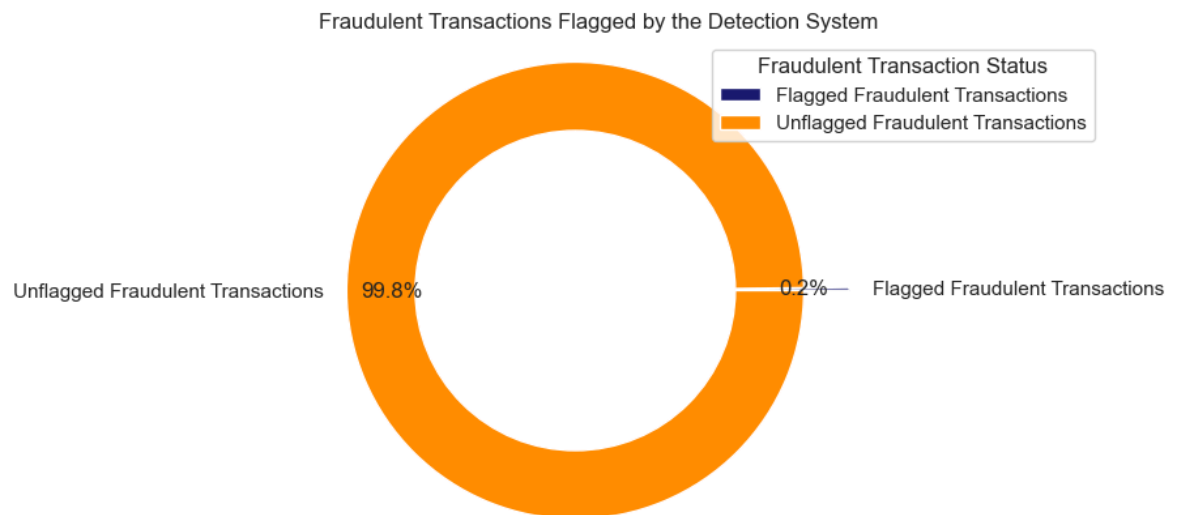
After analyzing the dataset, it is time to share what I learnt by showing some visualizations. Based on the 3 findings above, it is time to show share my findings with the use of graphs.

```
In [11]: ### Graph 1: Fraud Rate By Type (Bar Chart)
# Create the figure and axes
fig, ax = plt.subplots(nrows=1,ncols=1,figsize=(10,5));
# Create the bar graph
fraudRateTypeGraph=ax.bar(fraudTransactionFinding['Transaction type'], fraudTran
# Create the rest of the bar chart
ax.set(title='Fraud Rate by Transaction Type',ylabel='Fraud Rate (%)',xlabel='Tr
ax.bar_label(fraudRateTypeGraph,label_type='edge');
ax.grid(True);
plt.tight_layout();
plt.show();
```



As shown in the bar graph, we can see that transfer is the most likely transaction type for fraud to take place, followed by cash out. The remaining transaction types recorded no fraudulent transactions. We can also see that transfer is around 4 times more likely to have fraudulent transactions compared to cash out, which is supported in the first finding in the analyze phase.

```
In [12]: ### Graph 2: Fraud Detection System Accuracy(Donut Chart)
# Find the number of flagged and unflagged fraudulent transactions
flaggedFraudulentTransactions=df_fraud[df_fraud['isflaggedfraud']==1]['isflagged']
unflaggedFraudulentTransactions=df_fraud[df_fraud['isflaggedfraud']==0]['isflagged']
# Create the figure and axes
fig, ax = plt.subplots(nrows=1,ncols=1,figsize=(10,5));
# Create labels for the donut chart
label_text=['Flagged Fraudulent Transactions','Unflagged Fraudulent Transactions']
colors=['midnightblue','darkorange'];
explosion=[0.2,0.0];
# Create the donut graph
fraudDetectionAccuracyGraph=ax.pie([flaggedFraudulentTransactions,unflaggedFraudulentTransactions],
centre_circle=plt.Circle((0, 0), 0.70, fc='white'));
ax.add_artist(centre_circle)
ax.axis('equal');
# Create the rest of the donut chart
ax.set_title('Fraudulent Transactions Flagged by the Detection System');
plt.legend(title='Fraudulent Transaction Status',labels=label_text);
plt.show();
```



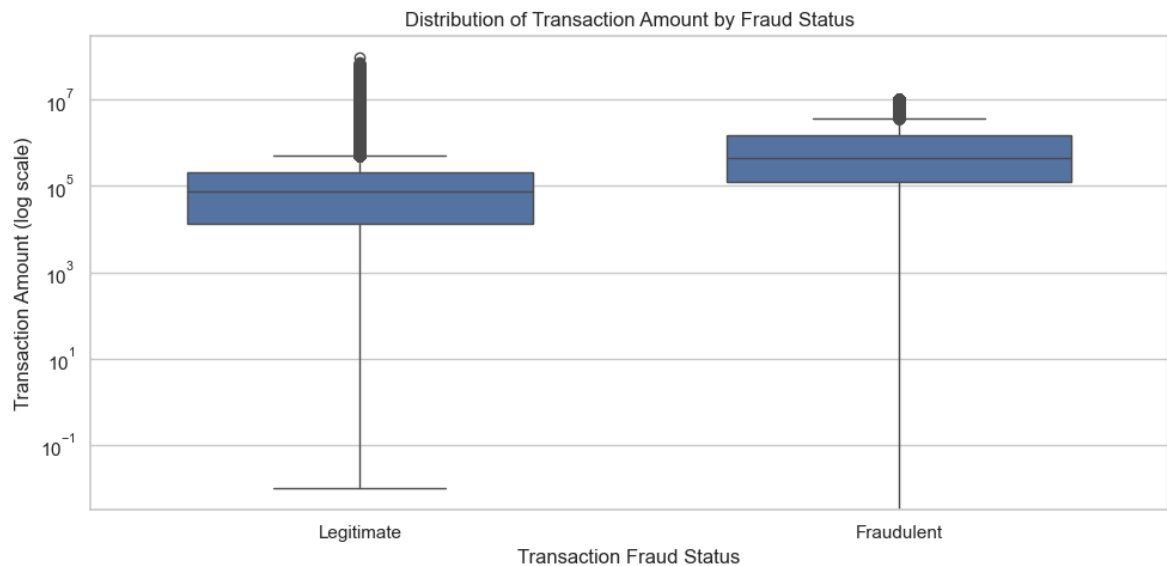
```
In [13]: flaggedFraudCount=pd.read_sql('SELECT COUNT(*) FROM financial_analysis WHERE isf
print(f'The number of fraudulent transactions that were flagged is {flaggedFraud
unflaggedFraudCount=pd.read_sql('SELECT COUNT(*) FROM financial_analysis WHERE i
print(f'The number of fraudulent transactions that were not flagged is {unflagge
```

The number of fraudulent transactions that were flagged is 16

The number of fraudulent transactions that were not flagged is 8197

For the donut graph, it tells us that the fraud flagging system currently in place performed poorly in finding fraudulent transactions as it is not able to find almost all of the fraudulent transactions in the dataset, indicating that its detection logic needs to be improved. This is supplemented by the sql queries I made after the graph, where just 16 out of the total 8213 fraudulent transactions were flagged as fraud, which showcases a critical vulnerability in the system, which is backed up with my second finding in the analyze phase.

```
In [14]: ### Graph 3: Fraud and Transaction Amount Distribution(Box plot)
sns.set_theme(style='whitegrid');
# Create the figure and axes
plt.figure(figsize=(10,5));
# Create the box plot
sns.boxplot(data=df_table, x='isfraud', y='amount', gap=0.2);
# Create the rest of the boxplot
plt.title('Distribution of Transaction Amount by Fraud Status');
plt.xlabel('Transaction Fraud Status');
plt.xticks([0, 1], ['Legitimate', 'Fraudulent']);
plt.ylabel('Transaction Amount (log scale)');
plt.yscale('log'); # reduce the effect of extreme balance values to make the gra
plt.tight_layout();
plt.show();
```



In the boxplot, we can see that legitimate transactions have a more varied range of the amount taken from the balance. For the fraudulent transactions, we can see that it is more consistent and more often larger than legitimate transactions. This shows that this can be a useful sign to identify fraudulent activities since fraudulent transactions don't occur across the entire transaction range of values. This complements my 3rd finding that fraudulent transactions are more likely to empty their balance, where both lead to the same pattern: fraudulent transactions tend to have large amounts of money and the balance gets emptied after the transaction has taken place.

The 3 findings relate to the original question of which payment transactions carry the most risk of fraud, and how fraudulent transactions can be identified based on their behaviours. We can see that Cash Outs and Transfer payments have the most amount of fraud and transactions that have large amounts of money and empty their balance tend to be fraudulent in nature so this behaviour can be monitored to better flag fraudulent transactions.

The Act Phase

In conclusion, as stated before, we can see that only cash out and transfer payments contain fraudulent transactions. We can also see that most of these frauds usually contain large amounts of money or they tend to empty their balance once the transaction has been completed. With these findings, we can more accurately pin down possible fraudulent transactions to review and improve the flagging system in order to protect our customers by inspecting cash out and transfer transactions that are over a specific amount or if the balance is empty after the transaction. One additional data point that can be used to better expand my findings is the use of a datetime stamp so analysis can be done based on specific times of the year, and also so that if these frauds take place, it can be tracked better.